Generating Case-Based Legal Arguments with LLMs

Morgan Gray*
Intelligent Systems Program,
University of Pittsburgh
Pittsburgh, Pennsylvania, USA
mag454@pitt.edu

Li Zhang
Intelligent Systems Program,
University of Pittsburgh
Pittsburgh, Pennsylvania, USA
liz239@pitt.edu

Kevin D. Ashley
Intelligent Systems Program,
University of Pittsburgh
Pittsburgh, Pennsylvania, USA
ashley@pitt.edu

Abstract

Over its decades long history, the field of AI and Law has made significant progress developing and researching formal models of case based reasoning that are capable of producing legal arguments. These models employ argument schemes to replicate legal argumentation. Although their arguments are accurate and explainable, these systems are costly to produce and maintain, requiring manual case representations and expert-crafted algorithms that mimic argument. To address these limitations we employ a prompt-engineering strategy that leads state-of-the-art LLMs to follow argument schemes. We show that it is feasible for LLMs to produce basic case-based legal arguments.

CCS Concepts

• **Computing methodologies → Natural language processing**;
• **Applied computing → Law**.

Keywords

Large Language Models, Legal Arguments, Factors, Legal Reasoning, Argument Scheme

ACM Reference Format:
Morgan Gray, Li Zhang, and Kevin D. Ashley. 2025. Generating Case-Based Legal Arguments with LLMs . In *Symposium on Computer Science and Law (CSLAW '25), March 25–27, 2025, München, Germany.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3709025.3712216

1 Introduction

We apply large language models (LLMs), generative artificial intelligence (GenAI), machine learning (ML) and natural language processing (NLP) in studying how to automatically implement and apply structured representations of legal concepts and argumentation in factor-based legal domains. Statutory or court-made law specifies factual considerations, that is, factors, that decision makers must or may take into account in determining diverse issues [23]. We believe that case-based reasoning could help a large language model (LLM) to explain predictions of case outcomes in terms that legal professionals can understand. In previous work, we have employed case-based reasoning methods to retrieve positive and negative case examples similar to an input case that can qualify a user's confidence in a machine learning program's prediction of an outcome [13]. Here we explore how well LLMs can generate case-based arguments and counterarguments that can further explain and qualify case outcome predictions using argument schemes, stereotypical patterns or templates of legal argument [11, 21].

Our investigation begins with a very basic argument scheme, 3-ply arguments comprising a point for one side, a response by the opponent, and a rebuttal by the initial side. In the point, a side, plaintiff or defendant, argues that an input case should be decided in its favor by drawing an analogy to a past case. In the response, the opposing side distinguishes the cited case and cites a counterexample that is as or more on point than the cited case. The initial side rebuts by distinguishing the counterexample.

In this work, we employ a collection of trade secret cases represented in terms of factors (defined below), translate the argument scheme into a prompt, randomly select sets of three cases that satisfy certain constraints, instruct three types of LLMs to generate 3-ply-arguments, and engage three legal experts in evaluating the resulting arguments using a grading rubric. We report and discuss the results here and describe next steps.

2 Related Work

The HYPO program [3] modeled reasoning with US trade secret law cases using factors, stereotypical patterns of fact that tend to strengthen or weaken a claim. It employed the factors to index cases and, as noted, to analogize and distinguish cases in 3-ply arguments. Aleven [2] organized factors into a factor hierarchy supporting more elaborated arguments in CATO. Rissland and Skalak [25] integrated legal rule-based reasoning with HYPO-style case-based arguments about the meanings of ill-defined statutory terms in a factor-based tax domain in CABARET. In the IBP program, Bruninghaus and Ashley [7] employed a model of trade secret law rules and issues that predicted outcomes of the individual issues and of the case as a whole. The AGATHA program [9] extended a factor-based model to address arguing with values underlying a property domain and trade secrets law. The VJAP program of Grabmair [12] modeled arguments analogizing cases based on decisions' effects on the underlying values. Horty developed a factor-based model of precedential constraint [16] for which Rigoni [24] suggested modifications.

The above methods (i.e., CATO [2], IBP [7], AGATHA [9], and VJAP [12]) all had pre-defined case databases of trade secret misappropriation cases (or a type of property case). The 100-200 trade secret cases were represented as lists of factors assigned by human readers, usually law students. Given a problem also represented as a list of factors, the programs, following an algorithm, retrieved relevant cases (i.e., those sharing at least one factor with the problem), organized them by relevance (e.g., by the inclusiveness of the overlaps of factors shared by the problem) and applied various

*All three authors contributed equally to this research.

This work is licensed under a Creative Commons Attribution 4.0 International License.
CSLAW '25, March 25–27, 2025, München, Germany
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1421-4/25/03
https://doi.org/10.1145/3709025.3712216

types of argument schemes to generate case-based arguments. Most of the argument schemes were more complex than the one illustrated in our paper; they modeled ways to emphasize or downplay a distinction or to account for value effects associated with the factors. Based on the strength of the resulting arguments, the above programs predicted outcomes.

These models would have practical utility but for two weaknesses: First, they relied on manually annotating cases' factors based on human coders' reading of the case texts. Second, it has not been clear how LLMs can generate legal arguments that correspond to these models' argument schemes in a way that would make them useful for explaining and critiquing ML-based case outcome predictions.

Data-driven methods based on large corpora of legal cases from, for example, the European Court of Human Rights, have been applied to predict outcomes of cases from their full-texts or sections thereof ([8], [1], [19], [28]). These approaches revealed a different weakness, however. They were not able to explain predictions in terms legal professionals can understand. Since they lacked a means of representing the concepts or structures of legal explanations, they could not fashion explanations of their predictions. By contrast, SMILE+IBP could explain its predictions in terms of the factors it had automatically identified in case texts [7]. It was the first program to predict outcomes of textually described cases, although the texts were case summaries, not their full texts.

The best that the full-text data-driven methods could do was to highlight portions of case texts that contributed the most weight to the predictions using hierarchical attention networks (HANs). See, e.g., [8]). Karl Branting's experiments with legal experts and non-experts suggested that such highlighted, high scoring portions of case texts do not amount to meaningful legal explanations [6]. Branting observed that the participants "had difficulty understanding the connection between the highlighted text and the issue that they were supposed to decide." Branting's SCALE project applied ML to predict case outcomes and to identify issues and factors in the texts of WIPO domain name dispute cases, both of which could be useful for explaining predictions [6]. Gray, et al. [13] automatically identified factors in drug interdiction auto stop (DIAS) cases, demonstrated their predictive value, and used ML techniques to explain case outcomes in terms legal professionals can understand. Following [26], Gray, et al. [13] [14] applied GPT-4o with zero-shot and few-shot learning automatically to annotate factors in DIAS cases. See also, [20] employing a transformer language model for the ascription of factors and issues in legal cases of the European Court of Human Rights.

The next step would be for LLMs to use factors in generating case-based arguments with which to explain and critique ML's predictions of case outcomes. Our work reported here assesses the feasibility of using case-based argument schemes to enable LLMs to generate such arguments.

Increasingly, AI and Law researchers are performing systematic experiments applying LLMs and GPT to legal tasks. The recent report of the LegalBench project addresses some types of legal reasoning that LLMs can perform [15]. The researchers apply LLMs with few-shot prompting to tasks like issue-spotting, rule-application, and interpretation. The LegalBench authors note [15, p. 28], however, that analogical reasoning grounded in case law is "an example of a reasoning ability which is not currently evaluated in LegalBench." Other work has focused on whether LLMs can reason analogically but not in a legal domain as we propose. See, for example [22] and [29] both of which address the task of generating analogous examples to support mathematical problem solving. Blass and Forbus [5] have developed a system to extract legal information from torts cases and apply the information to new cases by analogy and by converting generalizations of what the system has learned into logical rules. Unlike our proposed work, their legal domain is not factor-based, factors are not used to represent cases, and rule-based narrative functions, not LLMs, translate portions of the case texts into logical formalisms.

In empirical legal studies, researchers have applied decision trees on manually classified multi-factor cases, often with the aim of identifying which factors are most important, in a variety of legal domains such as 192 trademark infringement cases involving likelihood of consumer confusion as to the source of goods [4], about 800 child custody decisions [27], search and seizure decisions of the U.S. Supreme Court and appellate court confession cases [17], and 755 Canadian U.S. cases involving the employee / independent contractor distinction [10]. In addition to automating aspects of the factor annotation process, increasing efficiency and the number of analyzed decisions, our work here could add new ways to account for the strength of case-based arguments in the decisions.

3 Data

Following Ashley [3], we focus on the trade secret misappropriation domain. We rely on 149 annotated cases used in CATO [2]. Each case is represented by the factors that were applicable in the case and its outcome in favor of the plaintiff or defendant. With this information, we can model a simple argument scheme that involves citing an analogous case, responding by distinguishing and citing a counterexample, and rebutting by distinguishing the counterexample.

4 Methodology

Our central aim is to assess whether an LLM, namely, `gpt-4o`, `gpt-o1-preview`, and `llama3-70b`, can follow an argument scheme and produce an appropriate argument in accordance with the scheme. Formal models of case-based legal reasoning, such as CATO [2], IBP [7], AGATHA [9], and VJAP [12], have been shown to reliably produce arguments that follow particular argument schemes. As noted, however, these systems are costly to produce and maintain, having required manual case representations.

4.1 The Argument Scheme

The argument scheme that we are attempting to enforce is a 3-ply argument scheme [3]. The 3 represents the number of 'turns' or 'plies' of argument that are produced:

(1) Point for Initiator (plaintiff or defendant)
(2) Response for Responder (side opposing Initiator)
(3) Rebuttal for Initiator

The arguments are made about an "input scenario". An input scenario is one of the 149 annotated cases that serve as a potential factual scenario about which arguments can be made in favor of or against a court's finding. Here is an example of an input scenario:

Input Scenario:
short-name "Revcor" (outcome plaintiff)
Factors: #[1]
F1 Disclosure-in-negotiations (D)
*#F18 Identical-products (P)
*#F16 Info-reverse-engineerable (D)

The Initiator can represent either the plaintiff or defendant in the input scenario. If the Initiator represents the plaintiff, it argues the defendant should be liable for trade secret misappropriation. If the defendant is the initiator, it argues that it should not be liable for trade secret misappropriation.

In the first ply, the initiator makes an argument by analogy by:

(1) Citing a *citable* case and drawing an *analogy* between the input scenario and the cited case in terms of the set of factors they share. The initiator should also point out any additional factors in the input scenario that favor its position.

A **citable case** is one which was decided in favor of the side corresponding to the citer's side and which shares at least one factor with the citer's case that favors the citer's side [3]. An **analogy** between the citer's case and the cited case is the set of factors the citer's case shares with the cited case [3]. Here is an example of a citable case given the facts of Revcor:

Initiator's Case:
short-name "K & G" (outcome plaintiff)
F21 Knew-info-confidential (P)
F15 Unique-product (P)
F6 Security-measures (P)
*#F18 Identical-products (P)
F14 Restricted-materials-used (P)
*#F16 Info-reverse-engineerable (D)
F25 Info-reverse-engineered (D)

Note that this case meets the definition of a citable case because of the shared factor F18 Identical-products (P) and outcome, which favor the plaintiff, the Initiator in this example. An appropriate argument by the initiator (plaintiff) in this scenario looks like this:

Factors F18 Identical-products (P) and F16 Info-reverse-engineerable (D) are present in the citable case where the court found in favor of the plaintiff. There are no additional factors present in the input scenario that favor the Plaintiff.

In the second ply of the argument, the responder, will *distinguish* the case cited by the initiator, cite a *counterexample* in favor of the responder's position and draw an analogy between the counterexample and the input scenario. **Distinguishing** a cited case from the input case means reciting factors in the cited case that favor the citer's side that are not present in the input case and factors in the input case that favor the distinguisher's side that are not present in the cited case. A **counterexample** is a case for the responder such that the cited case's set of factors shared with the input case is a subset of, or the same set as, the counterexample's set of factors shared with the input case. Here is an example of a counterexample:

Responder's Case:
short-name "Speciner" (outcome defendant)
#F1 Disclosure-in-negotiations (D)
*#F18 Identical-products (P)
*#F16 Info-reverse-engineerable (D)

A proper argument in the second ply by the Responder (defendant) is:

The case cited by the Initiator is distinguishable. Factors F21 Knew-info-confidential (P), F15 Unique-product (P), F6 Security-measures (P), F14 Restricted-materials-used (P), were also present but are not present in the Input Scenario. F1 Disclosure-in-negotiations (D) is also present in the Input Scenario and favors the Responder. The Responder's Case is a counterexample to the Initiator's cited case. In the Responder's Case F1 Disclosure-in-negotiations (D), F18 Identical-products (P), and F16 Info-reverse-engineerable (D) were present and the court found in favor of the defendant.

In the third ply, the initiator rebuts the responder's case by distinguishing it from the input scenario. A proper rebuttal argument for the Initiator (plaintiff) is:

The case cited by the Responder is not distinguishable.

In summary, the argument scheme produces three plies where the Initiator draws an analogy, cites a case favorable to its position in the input scenario, and recites any additional factors in the input scenario that favor the Initiator. In the second ply, the responder distinguishes the cited case, recites any factors in the input scenario favorable to its position, and cites a counterexample to the cited case. In the third ply, the initiator has an opportunity to rebut the responder's argument by distinguishing the counterexample.[2]

4.2 Experimental Design

In order to provide the LLM with an input scenario, initiator's citing case, and responder's counterexample, we employ a randomized Monte Carlo algorithm. Because of the randomization, the algorithm may fail to identify a set of cases that satisfy the constraints of the 3-ply argument scheme. If this happens, the algorithm is run again until it returns a usable set of cases. During evaluation we verified that the output of this algorithm conformed to the constraints of the argument scheme. The algorithm randomly selects one of the 149 cases as the input scenario. The algorithm will then identify a case that fits the definition of a citable case on behalf of the Initiator. As noted, a citable case shares with the input scenario at least one factor that favor's the initiator's side. From there, it automatically identifies a counterexample such that the set of factors shared with the Input Scenario is a subset of, or the same set as, the counterexample's set of factors shared with the initator's citable case.[3]

[1]Here, # represents a matched factor between the input scenario and responder's counterexample. * represents a match between the input scenario and the initiator's cited case. *# is a match between the input scenario and both the initiator's cited case and the responder's counterexample.

[2]The prompts used in the experiment can be found at the following link: https://github.com/morganalexandergray/Generating-Case-Based-Legal-Arguments-with-LLMs-.git

[3]As an alternative methodology, we could present the LLM with a larger set of cases selected at random, and prompt it to make a 3-ply argument if possible. This, however, would require the LLM to perform extra work and could impact the results. Thus, to test the ability of the LLM to follow the argument scheme we provide suitable case triples.

We identified 15 sets of three cases to assess whether the LLM follows the argument scheme. In 10 sets, the cases are selected such that a proper 3-ply argument can be made in accordance with the argument scheme identified above. In these sets of cases, we use a "more on point" or "trumping counterexample", that is, a case whose outcome favors the responder and that shares a set of factors with the input scenario with respect to which the cited case's set of factors shared with the input scenario is a subset. In three of the 15 sets of cases, we use an "as on point" counterexample, that is, a case that shares the same set of factors as the input scenario and has an outcome in favor of the responder.

We hypothesize that if the LLM is actually performing reasoning, it should be able to detect whether the argument scheme cannot be followed. Accordingly, in two of the 15 sets of cases, the argument scheme could not be followed. We use these to assess how the model reacts when an argument cannot be made.

We evaluate the reasoning capability of three LLMs: `gpt-4o`, `llama3-70b`, and `gpt-o1-preview`. `gpt-4o` and `llama3-70b` are general purpose LLMs. By contrast, OpenAI recently released `gpt-o1-preview` for the avowed purpose of reasoning. For `gpt-4o` and `llama3-70b` we used greedy decoding, `Temperature=0`, `Top P=1`, `Max Tokens=500`, `Frequency Penalty=0`, and `Presence Penalty=0`. Since argumentation reasoning requires precision, we selected parameters to elicit a deterministic and highly probable response from the model. For `gpt-o1-preview`, we used the default parameters, which, at the time of writing are fixed by OpenAI at `Temperature=1`, `Top P=1`, `n=1` `Max Output Tokens=500`, `Frequency Penalty=0`, and `Presence Penalty=0`.

4.3 Prompt Design

To assess the feasabilty of this task and to enable comparisons between models, we use the same prompt for each LLM. We begin the prompt with an explanation and example of the legal problem of misappropriation of trade secrets:

> In this problem we are concerned with making legal arguments. We will make these arguments using factors important to trade secret misappropriation claims. Generally, the Plaintiff is alleging that the Defendant misappropriated its trade secret. For example, let's consider Kentucky Fried Chicken's original recipe. The original recipe is a blend of herbs and spices used to season fried chicken. If an employee working for Kentucky Fried Chicken published the recipe in a cookbook, Kentucky Fried Chicken (Plaintiff) could argue that the employee (Defendant) misappropriated its trade secret (the original recipe) by publishing it in a cookbook. Amongst other things, the Plaintiff might point to an agreement between the Plaintiff and the Defendant, where in the Defendant agreed not to disclose the recipe. The Defendant, might argue, however, that the defendant had already disclosed its trade secret (the original recipe) to an outsider and therefore, the Defendant should not be liable. Based on the factors described below, the Plaintiff will first make its argument and the Defendant will respond.

Next, we detail the factors that can be used for argumentation. Following Aleven [2], we detail the particular meaning of the factor and include a rationale for each factor and a brief statement explaining why the rationale supports the plaintiff's or defendant's position. The list of factors, meanings, and rationales is reproduced in Table 1.

Next, we reproduce the argument scheme almost identically to how it is presented in Section 4.1. Then we provide the model with an example input scenario, citable case, and counterexample along with a sample output. In the prompt we refer to the initator's citable case as TSC1 and the responder's counterexample as TSC2. We use the following example.

> In this scenario, the Initiator will be the Plaintiff and the Responder will be the Defendant. Here, the defendant makes a Trumping Counterexample.
>
> Example Input Scenario
> F1 Disclosure-in-negotiations (D)
> F4 Agreed-not-to-disclose (P)
> F6 Security-measures (P)
> F10 Secrets-disclosed-outsiders (D)
> F12 Outsider-disclosures-restricted (P)
> F14 Restricted-materials-used (P)
> F21 Knew-info-confidential (P)
>
> Example TSC 1
> outcome Plaintiff
> F4 Agreed-not-to-disclose (P)
> F6 Security-measures (P)
> F7 Brought-tools (P)
> F8 Competitive-advantage (P)
> F18 Identical-products (P)
>
> Example TSC 2
> outcome Defendant
> F3 Employee-sole-developer (D)
> F4 Agreed-not-to-disclose (P)
> F5 Agreement-not-specific (D)
> F6 Security-measures (P)
> F21 Knew-info-confidential (P)
>
> **Plaintiff's Argument:**
> Factors F4 Agreed-not-to-disclose (P) and F6 Security-measures (P) were present in TSC 1, where the court found in favor of the Plaintiff. In Addition, Factors F12 Outsider-disclosures-restricted (P), F14 Restricted-materials-used (P), F21 Knew-info-confidential (P) are present in the current factual scenario and favor the Plaintiff.
> **Defendant's Argument:**
> TSC1, cited by the plaintiff is distinguishable because factors F7 Brought-tools (P), F8 Competitive-advantage (P), and F18 Identical-products (P) were also present, but are not present in the Input Scenario. In addition,

Table 1: Factors and their corresponding meanings and rationales

Factor	Meaning	Rationale
F1 Disclosure-in-negotiations (D)	P disclosed its product information in negotiations with D	P gave his property away.
F2 Bribe-employee (P)	D paid P's former employee to switch employment, apparently in an attempt to induce the employee to bring P's information.	D obtained P's property through improper means.
F3 Employee-sole-developer (D)	Employee D was the sole developer of P's product.	D should have property rights in his invention.
F4 Agreed-not-to-disclose (P)	D entered into a nondisclosure agreement with P.	P takes reasonable steps to protect his property.
F5 Agreement-not-specific (D)	The nondisclosure agreement did not specify which information was to be treated as confidential.	P did not specify in what he claims a property interest.
F6 Security-measures (P)	P adopted security measures.	P takes reasonable steps to protect his property.
F7 Brought-tools (P)	P's former employee brought product development information to D.	D steals P's property.
F8 Competitive-advantage (P)	D's access to P's product information saved it time or expense.	P's trade secret is valuable property.
F10 Secrets-disclosed-outsiders (D)	P disclosed its product information to outsiders.	P gave his property away.
F11 Vertical-knowledge (D)	P's information is about customers and suppliers (which means that it may be available independently from customers or even in directories).	P cannot have a property interest in its customer's business info.
F12 Outsider-disclosures-restricted (P)	P's disclosures to outsiders were subject to confidentiality restrictions.	P protects his property.
F13 Noncompetition-agreement (P)	P and D entered into a noncompetition agreement.	P protected against former employee's use of confidential information.
F14 Restricted-materials-used (P)	D used materials that were subject to confidentiality restrictions.	D used P's property despite P's protections.
F15 Unique-product (P)	P was the only manufacturer making the product.	P's trade secret is valuable property.
F16 Info-reverse-engineerable (D)	P's product information could be learned by reverse-engineering.	P's property interest is limited in time.
F17 Info-independently-generated (D)	D developed its product by independent research.	P has no property interest in information D generated independently.
F18 Identical-products (P)	D's product was identical to P's.	D copied P's trade secret property.
F19 No-security-measures (D)	P did not adopt any security measures.	P did not protect his property.
F20 Info-known-to-competitors (D)	P's information was known to competitors.	P cannot have property interest in something known.
F21 Knew-info-confidential (P)	D knew that P's information was confidential.	D knew P claimed property interest.
F22 Invasive-techniques (P)	D used invasive techniques to gain access to P's information.	D used invasive techniques to steal P's property.
F23 Waiver-of-confidentiality (D)	P entered into an agreement waiving confidentiality.	P claimed no property interest in trade secret.
F24 Info-obtainable-elsewhere (D)	The information could be obtained from publicly available sources.	P cannot have property interest in something available from public sources.
F25 Info-reverse-engineered (D)	D discovered P's information through reverse engineering.	P's property interest is limited by time.
F26 Deception (P)	D obtained P's information through deception.	P was cheated of his property
F27 Disclosure-in-public-forum (D)	P disclosed its information in a public forum.	P gave his property interest in the trade secret away.

F1 Disclosure-in-negotiations (D) and F10 Secrets-disclosed-outsiders (D) are pro-defendant strengths present in the Input Scenario but not in TSC 1. TSC 2 is a counterexample to TSC 1. In TSC 2, F4 Agreed-not-to-disclose (P), F6 Security-measures (P), and F21 Knew-info-confidential (P) were present and the court found in favor of the Defendant.

Plaintiff's Rebuttal:

TSC2, cited by the Defendant is distinguishable. In TSC2, the additional factors F5 Agreement-not-specific (D) and F3 Employee-sole-developer (D) were present

and are not present here. Also, F12 Outsider-disclosures-restricted (P) and F14 Restricted-materials-used (P) are present in the input case but not in TSC 2.

Finally, we include a few instructions to control the behavior of the models. First, we instruct the model that it should only use the provided cases to draw analogies between the Input Scenario and the citable cases. Lastly, we instruct the model to follow the argument scheme in a formulaic manner and not to include extraneous text or explanations.

5 Evaluation

We rely on three legal experts to evaluate the ability of the model to follow the argument schemes. We evaluate each ply of the argument and the vital aspects of argument that should be present in each ply. In the first ply, we assess whether the Initiator makes an argument in favor of its position. The argument should (1) cite a citable case, (2) draw the analogy between the input case and the cited case, and (3) recite pro-side factors present in the input case. At the second ply we evaluate the responder's response to initiator's argument. The argument for the responder should (4) distinguish the case cited by the initiator, (5) recite pro-side factors present in the input case, (6) cite a counterexample, and (7) correctly analogize the counter example to the input case. In the third ply, we assess whether the initiator rebutted the Response. The initiator's rebuttal should (8) distinguish the case cited by the responder. For each of the eight criteria, we scored a 1 if the LLM followed the instructions and a 0 if it did not.

We evaluated the cases with sufficient overlap to calculate annotator agreement. Annotators 1 and 2 evaluated one set of 5 cases. Annotators 1 and 3 evaluated a different set of 5 cases. Annotators 2 and 3 evaluated yet another set of five cases. For purposes of evaluating the model in Table 3, we used the annotations of annotator 1 and 2 with annotator 1's scores taking precedence over annotator 2's in cases of conflict.

To evaluate annotator agreement, we assess the Cohen's Kappa score based on on the eight scoring criteria mentioned above. Each of the 3 annotators annotated 10 cases. Each annotator's 10 cases overlapped 5 cases of two other annotators. This enabled us to assess inter-annotator agreement between all annotators. The results of the annotator agreement assessment are in Table 2. Overall, the scores of Annotators 1, 2, and 3 are in substantial agreement. [18]. Agreement as to the quality of GPT-o1-Preview is the highest, and it is reflected in the model's high quality performance. By contrast, scoring disagreements arise regarding the lower performing GPT-4o and LlaMa3-70b models.

6 Results

Here, we examine how well the LLMs follow the 3-ply argument scheme. In Table 3, each criterion is examined individually to highlight the key aspects of the 3-ply argument structure. The calculations in Table 3 represent the average of the 1's and 0's recorded for each criterion across the 15 cases. Therefore, numbers close to 1 indicate high quality and numbers close to 0 indicate low quality on the part of the LLM in meeting that particular criterion. We do *not* include in our results the instances where the model was purposefully given cases where the argument scheme could not be followed. We discuss those two instances in the limitations section and error analysis.

In general, `gpt-o1-preview` performed very well. In six of the eight categories `gpt-o1-preview` was the highest performing model. In one of the eight categories it was tied for the highest performance. In one case, it was the second highest performing model. Generally, criteria 2, 4, 7 and 8 are the toughest to meet. `gpt-o1-preview` scored the highest on these criteria. From our results, it is apparent that the LLM constructed specifically for reasoning performs very well on the task even when using default parameters and without being thoroughly tuned. On the whole, `gpt-4o` and `llama3-70b` performed very similarly. The average scores in Table 3 reflect the similarity of the non-reasoning models. Here is an example of the output generated by GPT-o1-Preview:

> **Defendant's Argument:**
> Factors F19 No-security-measures (D) was present in TSC 1, where the court found in favor of the Defendant. In addition, Factors F1 Disclosure-in-negotiations (D) and F27 Disclosure-in-public-forum (D) are present in the current factual scenario and favor the Defendant.
> **Plaintiff's Argument:**
> TSC 1, cited by the Defendant, is distinguishable because factor F11 Vertical-knowledge (D) was present, favoring the Defendant, and is not present in the input scenario. Moreover, factor F18 Identical-products (P), favoring the Plaintiff, is present in the scenario but not in TSC 1. TSC 2 is a counterexample to TSC 1. In TSC 2, factors F1 Disclosure-in-negotiations (D), F18 Identical-products (P), and F19 No-security-measures (D) were present, and the court found in favor of the Plaintiff.
> **Defendant's Rebuttal:**
> TSC 2, cited by the Plaintiff, is distinguishable. In TSC 2, additional factors F8 Competitive-advantage (P) and F15 Unique-product (P) were present, favoring the Plaintiff, and are not present here. Also, factor F27 Disclosure-in-public-forum (D) is present in the input case but not in TSC 2.

7 Discussion

Our results demonstrate that it is feasible for an LLM to follow a legal argument scheme very closely, especially an LLM that has been trained for reasoning. Clearly, the reasoning capability of `gpt-o1-preview` shines through on this task. On the other hand, one sees that the utility of an LLM to generate and follow this simple form of legal reasoning is subject to limitations. Although there is no guarantee that a lawyer will perfectly follow any particular argument scheme, this is one of the strengths of formal models of case-based reasoning. Systems such as HYPO [3] are designed to faithfully follow an argument scheme in a reliable, repeatable, and explainable manner.

Table 2: Annotator Agreement Across Samples and Models

Sample & Annotator Pair	LLaMA3-70B	GPT-4o	GPT-o1-Preview	Average
Case Sample 1 [1,2]	0.69	0.78	1.00	0.82
Case Sample 2 [1,3]	0.58	0.49	0.90	0.65
Case Sample 3 [2,3]	0.57	0.62	1.00	0.73

Table 3: LLMs' Performance in Following the 3-Ply Argument Scheme

	LLaMA3-70B	GPT-4o	GPT-o1-Preview
Criterion 1: Cite a citable case.	1.00	0.92	1.00
Criterion 2: Draw an analogy between the input case and the cited case.	0.53	0.54	0.92
Criterion 3: Recite pro-side factors present in the input case.	0.77	0.62	0.92
Criterion 4: Distinguish the case cited by the Initiator.	0.38	0.54	0.92
Criterion 5: Recite pro-side factors present in the input case (for the Responder).	0.69	0.85	1.00
Criterion 6: Cite a counterexample.	1.00	0.92	1.00
Criterion 7: Correctly analogize the counterexample to the input case.	0.54	0.69	0.92
Criterion 8: Distinguish the case cited by the Responder.	0.15	0.38	1.00
Average across all criteria.	0.64	0.68	0.97

7.1 Error Analysis

To understand the limitations and necessary areas for improvement, we carefully examine and report the errors of each model quantitatively and qualitatively. We consider each of the eight criteria individually.

7.1.1 Criterion 1: Cite a citable case. First, considering whether the model cited a citable case, we are simply trying to ascertain whether the model made reference to the citable case in its response. For all models this was not a problem. In a notable instance, GPT-4o failed to properly reference a citable case. After reviewing the output, it appeared that the model confused the clearly labeled case that it was provided with the counterexample, which, unaccountably, it used as the initator's citable case.

7.1.2 Criterion 2: Draw an analogy between the input case and the cited case. Second, in considering whether the model draws the proper analogy between the input scenario and initiator's citable case, `gpt-01-preview` outperforms the others. In one instance, the model failed to draw the analogy between all possible factors. The input case was represented by the following factors: F1 Disclosure-in-negotiations (D), F18 Identical-products (P), F16 Info-reverse-engineerable (D). The initiator was the plaintiff. In the citable case the analogy could have been drawn with F18 Identical-products (P) and F16 Info-reverse-engineerable (D). The model only referenced F18 Identical-products (P). It seems that the model may have been confused and only referenced a pro-plaintiff factor.

GPT-4o and LLaMa3-70b erred more frequently. In some instances, the models missed a factor that could have been used in the analogy. In other instances, both models invented and made references to factors that were not shared between the input scenario and the citable case; the referenced factor was either missing from the input scenario or the citable case. In the failures made by the non-reasoning models, it seems that the model simply outputs likely factors given the context window without sufficient regard to the argument scheme.

7.1.3 Criterion 3: Recite pro-side factors present in the input case. Third, the model is then asked to recite additional factors present in the input scenario that favor the initiator's side, that is, the plaintiff should cite the factors present in the input scenario that favor the plaintiff and that have not already been included in the analogy with the cited case. This task seems simpler than drawing an analogy but more difficult than referencing a citable case. Here, `gpt-o1-preview` made a single error and missed one factor on one occasion. In some instances, the errors made by GPT-4o and LLaMa3-70b were fairly 'harmless' in that they would not have greatly impacted the argument, but still they did not follow the argument scheme. In one instance, GPT-4o repeated a factor that it had already used in analogizing and did not need to repeat. In other instances, LLaMa3-70b produced and referenced factors that were not present in the input scenario. This kind of error is more serious. In comparison if a lawyer were to simply recite a factor that it had already used in analogizing, no harm would be done except for slight redundancy. Arguing by referencing a factor that is not present is more problematic and could violate an advocate's duty of candor. In other instances, GPT-4o, for example, did not recognize an applicable factor and did not recite it in favor of its position.

7.1.4 Criterion 4: Distinguish the case cited by the initiator. Criterion 4 is focused on the second ply of the argument when the responder has an opportunity to make its case. In this instance, the model should distinguish the case cited by the initiator. The model must recognize meaningful differences between the input scenario and the case cited by the initiator. GPT-o1-Preview failed in one

scenario to distinguish all distinguishable factors. In other words, it could have referenced another factor favorable to it and present in the input scenario or one favorable to the initiator in the cited case. GPT-4o and LLaMa3-70b made the same mistake rather frequently and typically with reference to F16 Info-reverse-engineerable (D).

7.1.5 Criterion 5: Recite additional pro-side factors (for the responder) present in the input case. GPT-o1-Preview did not fail in this task. The non-reasoning models struggled with this task to a limited degree, with LLaMa3-70b erring more frequently than GPT-4o. In some instances, LLaMa3-70b cited factors as a strengths for the responder that were not in the input scenario. In another instance, the model confused the input scenario with the cited case. GPT-4o repeated redundant factors that it had already employed in analogizing. In one instance GPT-4o confusedly made an argument in favor of the defendant when it should have argued in favor of the plaintiff.

7.1.6 Criterion 6: Cite a counterexample. Sixth, we consider the models' ability to cite a counterexample. Much like the first criterion, the model must simply make reference to a counterexample that it has been provided. In the single instance where GPT-4o failed to do this correctly, the model confused the initiator's citable case for the responder's counterexample.

7.1.7 Criterion 7: Correctly analogize the counterexample to the input case. Seventh, we address the models' ability to correctly analogize the counterexample and the input case. In the single instance where GPT-o1-Preview failed to accomplish this task, the model missed a single factor that it could have used to analogize the counterexample and the input scenario. In many cases GPT-4o and LLaMa3-70b made the same mistake and missed a factor that it should have used in analogizing the input case and counterexample. In one instance, GPT-4o analogized the counterexample and the cited case, not the input scenario. In four instances LLaMa3-70b referenced factors that were not present in the input scenario. LLaMa3-70b struggled with this task more frequently than GPT-4o, even though it performed similarly to GPT-4o in analogizing the input scenario and the initiator's citable case.

7.1.8 Criterion 8: Distinguish the case cited by the responder. Lastly, in criterion 8, we examine the capability of the model to perform the rebuttal argument. GPT-o1-Preview again performed perfectly at this task. This model seemed to be able to reason about when a rebuttal was possible and when it was not. In some instances, the case cited by the responder cannot be rebutted by the initiator. In one positive example, the model returned the following rebuttal:

> TSC2 cited by the Plaintiff, is indistinguishable from the input scenario based on factors, but given that the only factor present favors the Defendant the court should find in favor of Defendant as in TSC1.

In another example, it noted an as-on-point counterexample, that is, the counterexample and cited case shared the same factors with the Input Scenario. In this instance, the model responded:

> TSC2 cited by the Plaintiff, cannot be distinguished from the input scenario, as all factors present in TSC2 are also present in the input scenario.

This output is even more impressive given the fact that we do not specifically instruct the model what to do in a case where rebuttal is not possible. This appears to show some inherent reasoning capability in the model without direct instruction. On the other hand, both GPT-4o and LLaMa3-70b generally attempted to force the pattern of the 3-ply argument even if it did not make sense. In two instances, LLaMa3-70b correctly pointed out that a case was not distinguishable on rebuttal, but still followed the form of the argument by beginning its response with "TSC2, cited by the Plaintiff is distinguishable". In some instances LLaMa3-70b made a rebutting argument with non-applicable factors. In other instances, the model missed factors that could have been distinguished. Another output shows that the model made irrelevant references to the initiator's citable case. GPT-4o suffered from similar problems and tried to distinguish factors that did not meet the definition of distinctions. In one instance, GPT-4o recognized that no rebuttal could be made and tried to make an argument by reiterating the counterexample.

> TSC2, cited by the Plaintiff, is distinguishable. In TSC2, there are no additional factors that favor the Plaintiff which are not present in the Input Scenario. However, the presence of F1 Disclosure-in-negotiations *D* and F16 Info-reverse-engineerable*D* in both TSC 2 and the Input Scenario strongly supports the Defendant's position.

7.1.9 Incorrect Case Information. The final aspect of our investigation focuses on the models' behavior fo two sets of cases where a 3-ply argument could not be made with the case information provided to the model. We use these examples to assess how well the model could understand and appear to reason with the input. All three models tried to follow the argument scheme and produced output that does not make sense in the context of the argument scheme. None of the models' reasoning capability extends to understanding when the argument scheme cannot be followed.

In sum, we show that the reasoning model GPT-o1-Preview shows near perfect performance in dutifully following and applying the argument scheme set forth in the prompt. Our results are not to say, however, that LLaMa3-70b and GPT-4o are incapable after serious efforts at prompt tuning, further prompt instruction, etc. Overall, in our review of the output of LaMa3-70b and GPT-4o we observed the models generated output that was probabilistically likely given the information in the prompt, but that output alone is insufficient even to capture simple legal reasoning such as a 3-ply argument. Moreover, the errors made by GPT-o1-Preview were often because the model did not generate the particular component of the argument completely, such as missing an applicable factor.

8 Limitations

Our goal was to evaluate LLMs' ability to follow an argument scheme when provided with the proper information to do so. Our sample cases were randomly selected although the number of samples was limited (15 samples of argument). Another limitation is that we provided the models with the information needed to make the arguments including the cases, their outcomes, the applicable factors and which side each factor favored, plaintiff or defendant. To varying extents, the models appear to be able to reason with this information well enough to follow an argument scheme's template.

Further exploration is needed to assess how well the models can follow an argument scheme where the cases are expressed textually and a model needs to decide which factors apply. Our use of a mix of proprietary and open source models is also a limitation. The proprietary reasoning model GPT-o1-Preview performed the best, but these models are known to provide limited research utility due to their frequent changes and unspecified training sets. Furthermore, the evaluation of the model's output is a time intensive and heavily manual task. We look forward in future work to potentially fine-tuning an LLM for the purpose of evaluating output.

9 Conclusions and Future Work

Our goal in this experiment was to determine if LLMs could apply a simple argument scheme that CATO (or its forerunner, HYPO) applied as well as CATO would apply it. In effect, this is what the the human evaluators in our paper were manually determining: whether the LLM was correctly applying CATO's argument scheme. Our results show that LLMs can employ a simple argument scheme through prompts that elicit legal reasoning via few-shot in-context learning.

In future work, we will attempt to generate more robust, text based arguments, rather than rely solely on factor labels. We will also address how well an LLM can implement more complex argument schemes including schemes for emphasizing or downplaying distinctions [2] and for arguing about a proposed decision's effect on values underlying legal issues [12].

Our results suggest that it is feasible eventually to elicit the same kind of reasoning that would be employed in more complex formal models of computational reasoning. LLMs that perform this kind of reasoning could be developed and maintained much more quickly and with less cost to than symbolic models of reasoning. LLMs purpose-built for reasoning appear to mimic human reasoning at a much higher level than other state-of-the-art LLMs. In light of related work in which LLMs can automatically annotate factors in case texts, these results suggest that LLMs may overcome the limitations of formal case-based methods of legal reasoning that rely on manually annotated case representations.

The potential for applying LLMs to then make arguments with such cases is intriguing. Potentially, we would no longer be limited to a few hundred manually represented cases. Eventually, this work may augment machine learning based outcome predictions with explanations and arguments legal practitioners can understand.

References

[1] Nikolaos Aletras, Dimitrios Tsarapatsanis, Daniel Preoţiuc-Pietro, and Vasileios Lampos. 2016.

[2] Vincent Aleven. 1997. *Teaching case-based argumentation through a model and examples.* University of Pittsburgh.

[3] Kevin D Ashley. 1990. *Modeling Legal Arguments: Reasoning with Cases and Hypotheticals.* MIT Press.

[4] Barton Beebe. 2006. An empirical study of the multifactor tests for trademark infringement. *Calif. L. Rev.* 94 (2006), 1581–1654.

[5] Joseph Blass and Kenneth D Forbus. 2023. Analogical Reasoning, Generalization, and Rule Learning for Common Law Reasoning. In *Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law.* 32–41.

[6] L Karl Branting, Craig Pfeifer, Bradford Brown, Lisa Ferro, John Aberdeen, Brandy Weiss, Mark Pfaff, and Bill Liao. 2021. Scalable and explainable legal prediction. *Artificial Intelligence and Law* 29, 2 (2021), 213–238.

[7] Stefanie Brüninghaus and Kevin D Ashley. 2003. Predicting outcomes of case based legal arguments. In *Proceedings of the 9th International conference on Artificial Intelligence and Law.* ACM, 233–242.

[8] Ilias Chalkidis, Ion Androutsopoulos, and Nikolaos Aletras. 2019. Neural legal judgment prediction in English. *arXiv preprint arXiv:1906.02059* (2019).

[9] Alison Chorley and Trevor Bench-Capon. 2005. AGATHA: Using heuristic search to automate the construction of case law theories. *Artificial Intelligence and Law* 13, 1 (2005), 9–51.

[10] Maxime C Cohen, Samuel Dahan, Warut Khern-Am-Nuai, Hajime Shimao, and Jonathan Touboul. 2023. The use of AI in legal systems: determining independent contractor vs. employee status. *Artificial intelligence and law* (2023), 1–30.

[11] Thomas F Gordon and Douglas Walton. 2009. Legal reasoning with argumentation schemes. In *Proceedings of the 12th international conference on artificial intelligence and law.* 137–146.

[12] Matthias Grabmair. 2017. Predicting trade secret case outcomes using argument schemes and learned quantitative value effect tradeoffs. In *Proceedings of the 16th ICAIL.* 89–98.

[13] Morgan Gray, Jaromir Savelka, Wesley Oliver, and Kevin Ashley. 2023. Automatic Identification and Empirical Analysis of Legally Relevant Factors. In *International Conference of Artificial Intelligence and Law (ICAIL 2023).* ACM Press.

[14] Morgan Gray, Jaromir Savelka, Wesley Oliver, and Kevin Ashley. 2024. Empirical legal analysis simplified: reducing complexity through automatic identification and evaluation of legally relevant factors. Issue 2270. https://doi.org/10.1098/rsta.2023.0155

[15] Neel Guha, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel Rockmore, Diego Zambrano, et al. 2024. Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models. *Advances in Neural Information Processing Systems* 36 (2024).

[16] John Horty. 2021. Modifying the reason model. *Artificial Intelligence and Law* 29 (2021), 271–285.

[17] Jonathan P. Kastellec. 2010. The Statistical Analysis of Judicial Decisions and Legal Rules with Classification Trees. *Journal of Empirical Legal Studies* 7, 2 (2010), 202–230. https://doi.org/10.1111/j.1740-1461.2010.01176.x arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1740-1461.2010.01176.x

[18] Mary L. McHugh. 2012. Interrater reliability: the kappa statistic. , 276-82 pages.

[19] Masha Medvedeva, Martijn Wieling, and Michel Vols. 2022. Rethinking the field of automatic prediction of court decisions. *Artificial Intelligence and Law* (2022), 1–18.

[20] Jack Mumford, Katie Atkinson, and Trevor Bench-Capon. 2023. Combining a legal knowledge model with machine learning for reasoning with legal cases. In *Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law.* 167–176.

[21] Henry Prakken. 2005. AI & Law, logic and argument schemes. *Argumentation* 19 (2005), 303–320.

[22] Chengwei Qin, Wenhan Xia, Tan Wang, Fangkai Jiao, Yuchen Hu, Bosheng Ding, Ruirui Chen, and Shafiq Joty. 2024. Relevant or Random: Can LLMs Truly Perform Analogical Reasoning? *arXiv preprint arXiv:2404.12728* (2024).

[23] Scott Rempell. 2022. Factors. *Buff. L. Rev.* 70 (2022), 1–57. http://dx.doi.org/10.2139/ssrn.4095435

[24] A Rigoni. 2015. An improved factor based approach to precedential constraint. *Artificial Intelligence and Law* 23, 2 (2015), 133–160.

[25] Edwina L Rissland and David B Skalak. 1991. Cabaret:rule interpretation in a hybrid architecture. *International Journal of Man-machine Studies* 34 (1991), 839–887.

[26] Jaromir Savelka and Kevin D Ashley. 2023. The unreasonable effectiveness of large language models in zero-shot semantic annotation of legal texts. *Frontiers in Artificial Intelligence* 6 (2023).

[27] Hsuan-Lei Shao, Robert B Leflar, and Sieh-Chuen Huang. 2022. Factors Determining Child Custody in Taiwan after Patriarchy's Decline: Decision Tree Analysis on Family Court Decisions. *Asian J. of Comparative Law* (2022), 1–17.

[28] Santosh Tyss, Marcel Perez San Blas, Phillip Kemper, and Matthias Grabmair. 2023. Leveraging task dependency and contrastive learning for case outcome classification on european court of human rights cases. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics.* 1103–1111.

[29] Michihiro Yasunaga, Xinyun Chen, Yujia Li, Panupong Pasupat, Jure Leskovec, Percy Liang, Ed H Chi, and Denny Zhou. 2023. Large language models as analogical reasoners. *arXiv preprint arXiv:2310.01714* (2023).

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009